

Esercizio 2: Server Linux

Executive Summary

Questo documento descrive i passaggi necessari per monitorare i server in esecuzione sul sistema Linux tramite interfaccia a riga di comando.

Domanda 1:

Perché è stato necessario eseguire ps come root (premettendo il comando con sudo)?

Perché non tutte le cartelle vengono mostrate dato che alcune sono accessibili solo con i privilegi di root.

```
[analyst@secOps ~]$ sudo ps -elf
[sudo] password for analyst:
F S UID          PID     PPID  C  PRI  NI ADDR SZ WCHAN  STIME TTY          TIME CMD
4 S root           1         0  0   80   0 -   5503 do_epo 06:47 ?        00:00:00 /sbin/init
1 S root           2         0  0   80   0 -           0 kthrea 06:47 ?        00:00:00 [kthreadd]
1 S root           3         2  0   80   0 -           0 kthrea 06:47 ?        00:00:00 [pool_workqueue_release]
1 I root           4         2  0   60  -20 -           0 rescue 06:47 ?        00:00:00 [kworker/R-rcu_gp]
1 I root           5         2  0   60  -20 -           0 rescue 06:47 ?        00:00:00 [kworker/R-sync_wq]
1 I root           6         2  0   60  -20 -           0 rescue 06:47 ?        00:00:00 [kworker/R-kvfree_rcu_reclaim]
1 I root           7         2  0   60  -20 -           0 rescue 06:47 ?        00:00:00 [kworker/R-slub_flushwq]
1 I root           8         2  0   60  -20 -           0 rescue 06:47 ?        00:00:00 [kworker/R-netns]
1 I root           9         2  0   80   0 -           0 worker 06:47 ?        00:00:00 [kworker/0:0-events]
1 I root          10         2  0   80   0 -           0 worker 06:47 ?        00:00:00 [kworker/0:1-events]
1 I root          12         2  0   80   0 -           0 worker 06:47 ?        00:00:00 [kworker/u8:0-events_power_efficient]
5 I root          13         2  0   80   0 -           0 worker 06:47 ?        00:00:00 [kworker/u8:1-events_unbound]
1 I root          14         2  0   60  -20 -           0 rescue 06:47 ?        00:00:00 [kworker/R-mm_percpu_wq]
1 S root          15         2  0   80   0 -           0 smpboo 06:47 ?        00:00:00 [ksoftirqd/0]
1 I root          16         2  0   58   - -           0 rcu_gp  06:47 ?        00:00:00 [rcu_preempt]
1 S root          17         2  0   58   - -           0 rcu_bo  06:47 ?        00:00:00 [rcub/0]
1 S root          18         2  0   80   0 -           0 kthrea 06:47 ?        00:00:00 [rcu_exp_par_gp_kthread_worker/0]
1 S root          19         2  0   80   0 -           0 kthrea 06:47 ?        00:00:00 [rcu_exp_gp_kthread_worker]
1 S root          20         2  0  -40   - -           0 smpboo 06:47 ?        00:00:00 [migration/0]
1 S root          21         2  0    9   - -           0 smpboo 06:47 ?        00:00:00 [idle_inject/0]
1 S root          22         2  0   80   0 -           0 smpboo 06:47 ?        00:00:00 [cpuhp/0]
```

Domanda 2:

Come viene rappresentata la gerarchia dei processi da ps?

Viene rappresentata tramite il **PID** (*Process ID*), affidato al processo in ordine cronologico per apertura del processo.

```
[analyst@secOps ~]$ sudo ps -elf
[sudo] password for analyst:
F S UID          PID     PPID  C  PRI  NI ADDR SZ WCHAN  STIME TTY          TIME CMD
1 S root           1         0  0   80   0 -  5503 do_epo 06:47 ?        00:00:00 /sbin/init
1 S root           2         0  0   80   0 -    0 kthrea 06:47 ?        00:00:00 [kthreadd]
1 S root           3         2  0   80   0 -    0 kthrea 06:47 ?        00:00:00 [pool_workqueue_release]
1 I root           4         2  0   60 -20 -    0 rescue 06:47 ?        00:00:00 [kworker/R-rcu_gp]
1 I root           5         2  0   60 -20 -    0 rescue 06:47 ?        00:00:00 [kworker/R-sync_wq]
1 I root           6         2  0   60 -20 -    0 rescue 06:47 ?        00:00:00 [kworker/R-kvfree_rcu_reclaim]
1 I root           7         2  0   60 -20 -    0 rescue 06:47 ?        00:00:00 [kworker/R-slub_flushwq]
1 I root           8         2  0   60 -20 -    0 rescue 06:47 ?        00:00:00 [kworker/R-netns]
1 I root           9         2  0   80   0 -    0 worker 06:47 ?        00:00:00 [kworker/0:0-events]
1 I root          10         2  0   80   0 -    0 worker 06:47 ?        00:00:00 [kworker/0:1-events]
1 I root          12         2  0   80   0 -    0 worker 06:47 ?        00:00:00 [kworker/u8:0-events_power_efficient]
5 I root          13         2  0   80   0 -    0 worker 06:47 ?        00:00:00 [kworker/u8:1-events_unbound]
1 I root          14         2  0   60 -20 -    0 rescue 06:47 ?        00:00:00 [kworker/R-mm_percpu_wq]
1 S root          15         2  0   80   0 -    0 smpboo 06:47 ?        00:00:00 [ksoftirqd/0]
1 I root          16         2  0   58   - -    0 rcu_gp  06:47 ?        00:00:00 [rcu_preempt]
1 S root          17         2  0   58   - -    0 rcu_bo  06:47 ?        00:00:00 [rcub/0]
1 S root          18         2  0   80   0 -    0 kthrea 06:47 ?        00:00:00 [rcu_exp_par_gp_kthread_worker/0]
1 S root          19         2  0   80   0 -    0 kthrea 06:47 ?        00:00:00 [rcu_exp_gp_kthread_worker]
1 S root          20         2  0  -40   - -    0 smpboo 06:47 ?        00:00:00 [migration/0]
```

Domanda 3:

Qual è il significato delle opzioni -t, -u, -n, -a e -p in netstat? (usa man netstat per rispondere):

```
<Socket>={-t|--tcp} {-u|--udp} {-U|--udplite} {-S|--sctp} {-w|--raw}
```

-t (--tcp): Mostra solo le connessioni che utilizzano il protocollo **TCP**.

-u (--udp): Mostra solo le connessioni che utilizzano il protocollo **UDP**.

```
--numeric, -n
    Show numerical addresses instead of trying to determine symbolic host, port or user names.
```

-n (--numeric): Mostra gli indirizzi IP e i numeri di porta in formato **numerico**, senza cercare di risolverli in nomi di host, nomi di porta o nomi utente (evita la risoluzione DNS).

```
-a, --all
    Show both listening and non-listening sockets. With the --interfaces option, show interfaces that are not up
```

-a (--all): Mostra **tutti** i socket, sia quelli in ascolto (listening) che quelli non in ascolto (connessioni attive/stabilite). Di default, netstat mostra solo i socket connessi, omettendo i server in ascolto.

```
-p, --program
    Show the PID and name of the program to which each socket belongs. A hyphen is shown if the socket belongs to the
    kernel (e.g. a kernel service, or the process has exited but the socket hasn't finished
    closing yet).
```

-p (--programs): Mostra l'ID del processo (PID) e il **nome del programma/processo** proprietario di ciascun socket.

Domanda 4:

L'ordine delle opzioni è importante per netstat?

In generale, l'ordine dei parametri in netstat non ha importanza. Le flag possono essere messe separate o anche assieme dopo un singolo "-". Puoi combinare le opzioni (flag) come preferisci, ad esempio netstat -ano è equivalente a netstat -a -n -o.

Se si utilizzano parametri che richiedono un valore a seguito (come -p per specificare un protocollo o -l per un'interfaccia), quel valore specifico deve seguire immediatamente l'opzione corrispondente.

Ad esempio: **netstat -p tcp** e non **netstat tcp -p**.

Domanda 5:

Basandosi sull'output di netstat mostrato al punto (d), qual è il protocollo di Livello 4, lo stato della connessione e il PID del processo in esecuzione sulla porta 80?

Protocollo: TCP

Stato: In ascolto

PID: 1464

```
[analyst@secOps ~]$ sudo netstat -tunap
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 0.0.0.0:6633            0.0.0.0:*               LISTEN      361/python3.9
tcp        0      0 0.0.0.0:22             0.0.0.0:*               LISTEN      376/sshd: /usr/bin/
tcp        0      0 0.0.0.0:21             0.0.0.0:*               LISTEN      423/vsftpd
tcp        0      0 0.0.0.0:80             0.0.0.0:*               LISTEN      1464/nginx: master
tcp6       0      0 :::22                  :::*                    LISTEN      376/sshd: /usr/bin/
udp        0      0 10.0.2.15:68           0.0.0.0:*                287/systemd-network
```

Domanda 6:

Sebbene i numeri di porta siano solo una convenzione, puoi indovinare che tipo di servizio è in esecuzione sulla porta 80 TCP?

Sulla porta 80 gira il servizio **nginx**.

Domanda 7:

Il processo PID 395 è nginx. Come si potrebbe concludere questo dall'output sopra?

Perché il PID 395 è associato al **Master Process**.

Domanda 8:

Cos'è nginx? Qual è la sua funzione? Usa google per saperne di più su nginx)

NGINX è un web server open-source leggero e ad alte prestazioni, utilizzato anche come reverse proxy, bilanciatore di carico (load balancer) e cache HTTP. Progettato per gestire migliaia di connessioni simultanee con un basso utilizzo di risorse, è ideale per siti ad alto traffico.

Funzioni principali di NGINX:

- **Web Server:** Serve contenuti statici (*HTML, immagini, CSS*) in modo estremamente veloce e gestisce applicazioni dinamiche.
- **Reverse Proxy:** Agisce da intermediario tra i client (*browser*) e i server backend, inoltrando le richieste, migliorando la sicurezza e gestendo la terminazione SSL/TLS.
- **Load Balancer (Bilanciatore di Carico):** Distribuisce il traffico in entrata su più server applicativi per ottimizzare le prestazioni e garantire la continuità del servizio (*High Availability*).
- **HTTP Cache:** Memorizza risposte temporanee per ridurre il carico sul server di origine e velocizzare i tempi di risposta per gli utenti.
- **Sicurezza:** Può fungere da Web Application Firewall (*WAF*) per proteggere da attacchi comuni come DDoS.

Domanda 9:

La seconda riga mostra che il processo 396 è di proprietà di un utente chiamato http e ha il processo numero 395 come processo genitore. Cosa significa? È un comportamento comune?

È un comportamento comune e principalmente serve a mantenere i processi separati e sicuri. Nginx avvia un master process in root per poter agire sulle porte privilegiate 80/443 e leggere file di configurazione e certificati SSL/TLS in maniera sicura.

Aperte le porte avvia poi un processo worker per interagire e dirigere il traffico web con utente http, così che se corrotto non avrebbe i privilegi di root.

Domanda 10:

Perché l'ultima riga mostra ``grep 395``?

Il comando **grep** appare tra i risultati poiché è un processo attivo (*PID 1189*) che contiene la stringa 395 nei suoi argomenti, quindi è un processo che va in esecuzione nel momento esatto in cui il sistema genera la lista dei processi.

Domanda Nascosta

Sebbene una rapida ricerca su internet abbia rivelato che **nginx** è un server web leggero, come potrebbe un analista esserne sicuro?

Un analista può verificarlo comunicando direttamente con il servizio attraverso una tecnica chiamata banner grabbing.

Come mostrato nell'immagine, utilizzando telnet per connettersi all'indirizzo IP locale sulla porta 80 (*la porta standard per il traffico HTTP*), l'analista può inviare dati al servizio. Anche inviando tasti casuali (come "*fdsafsdaf*"), il servizio risponde con un'intestazione HTTP. In questa risposta, il server stesso dichiara la sua identità nella riga:

Server: nginx/1.16.1

Questa conferma "diretta" dal servizio è molto più attendibile di una semplice ricerca su internet o del nome del processo nel task manager.

```
fdsafsdaf
HTTP/1.1 400 Bad Request
Server: nginx/1.16.1
Date: Tue, 28 Apr 2020 20:09:37 GMT
Content-Type: text/html
Content-Length: 173
Connection: close

<html>
<head><title>400 Bad Request</title></head>
<body bgcolor="white">
<center><h1>400 Bad Request</h1></center>
<hr><center>nginx/1.16.1</center>
</body>
</html>
Connection closed by foreign host.
```

Domanda Nascosta

E se un attaccante avesse cambiato il nome di un programma malware in **nginx**, solo per farlo sembrare il popolare webserver?

Se un malware fosse semplicemente rinominato "nginx", difficilmente si comporterebbe come un vero server web. Ecco cosa accadrebbe durante un test con telnet:

- **Comportamento anomalo:** Un malware potrebbe non rispondere affatto sulla porta 80, oppure potrebbe chiudere immediatamente la connessione se riceve dati che non si aspetta.
- **Risposta non conforme:** Se il malware risponde, probabilmente non restituirebbe intestazioni HTTP valide (**HTTP/1.1 400 Bad Request**) o una pagina HTML di errore formattata correttamente come quella mostrata nell'esempio.

Domanda 11:

Perché l'errore è stato inviato come pagina web?

L'errore è stato restituito sotto forma di pagina web perché Nginx presuppone di dialogare sempre con un browser web. Inviando testo casuale via Telnet, si viola la sintassi del protocollo HTTP che il server si aspetta di ricevere sulla porta 80. Di conseguenza, Nginx interpreta l'input come una richiesta malformata, genera un errore 400 Bad Request e lo formatta come pagina HTML, poiché è lo standard predefinito dei web server per mostrare gli errori agli utenti. Nginx non è consapevole del fatto che il client utilizzato sia un terminale a riga di comando anziché un browser.

Domanda 12:

Usa Telnet per connetterti alla porta 68. Cosa succede? Spiega.

L'esecuzione del comando telnet 127.0.0.1 68 restituisce l'errore "**Connection refused**". Questo indica che il sistema operativo ha respinto attivamente la richiesta poiché non vi è alcun servizio TCP in ascolto su tale porta. La porta 68 è infatti tipicamente riservata al client DHCP, un servizio che opera sul protocollo UDP; di conseguenza, il tentativo di connessione tramite Telnet (che utilizza esclusivamente il protocollo TCP) fallisce e viene immediatamente rifiutato dal sistema.

Telnet usa il protocollo TCP, mentre la porta 68 (DHCP) usa un protocollo UDP.

Domanda 13:

Quali sono i vantaggi dell'uso di netstat?

Permette di visualizzare connessioni attive e porte in ascolto, facilitando sia il controllo della sicurezza sia la risoluzione di problemi di rete.

1. Visibilità immediata delle connessioni attive

Netstat permette di visualizzare tutte le connessioni TCP e UDP attualmente attive sul sistema. In ambito cyber, questo è il primo passo per scovare traffico anomalo, come ad esempio un malware che sta comunicando con un server *Command & Control* (C2) esterno o un'esfiltrazione di dati in corso.

2. Identificazione delle porte in ascolto (Listening Ports)

Il comando mostra quali porte sono aperte e in attesa di connessioni in ingresso. Questo è essenziale per:

- Individuare servizi non necessari che aumentano la superficie di attacco.
- Scoprire *backdoor* o *bind shell* lasciate in ascolto da un attaccante per mantenere la persistenza nel sistema.

3. Mappatura Processo-Rete

Questo è forse il vantaggio più grande per un analista. Utilizzando flag specifici (come netstat -ano su Windows o netstat -tunlp su Linux), puoi collegare ogni connessione o porta in ascolto al PID (Process ID) del programma che l'ha generata.

4. Analisi delle statistiche e del routing

Oltre alle connessioni, netstat può mostrare la tabella di routing del sistema (con il flag -r) e le statistiche dettagliate delle interfacce di rete (con il flag -s o -i).

Domanda 14:

Quali sono i vantaggi dell'uso di Telnet ? È sicuro ?

I suoi vantaggi sono limitati a casi d'uso molto specifici:

- **Troubleshooting di rete:** È un ottimo strumento per verificare se una porta specifica su un server remoto è aperta. Puoi usare la sintassi telnet [indirizzo_IP] [porta]. Se la connessione viene stabilita, sai che un servizio è in ascolto su quella porta, indipendentemente dal protocollo reale.
- **Supporto per dispositivi Legacy:** Alcuni vecchi router, switch o sistemi industriali (SCADA/ICS) che non hanno potenza di calcolo sufficiente per la crittografia, supportano solo Telnet per l'amministrazione remota.
- **Leggerezza e Semplicità:** Richiede pochissime risorse di sistema, motivo per cui veniva largamente utilizzato prima che i processori fossero in grado di gestire facilmente calcoli crittografici complessi.

Non è sicuro: Il problema fondamentale di Telnet è che trasmette tutte le comunicazioni in chiaro (plaintext). Questo significa che i nomi utente, le password e tutti i comandi inviati tra il tuo client e il server viaggiano sulla rete senza alcuna crittografia. Se un attaccante si trova sulla tua stessa rete (come nel caso di una compromissione locale o di un attacco Man-in-the-Middle), può catturare e leggere l'intero traffico senza sforzo.